



System design

Chat App (e.g. Messenger) ★ Premium



Hard



30 mins



849 done

Report an issue 🛠️

Asked at these companies



Airbnb



OpenAI



Anthropic



Discord



Snap

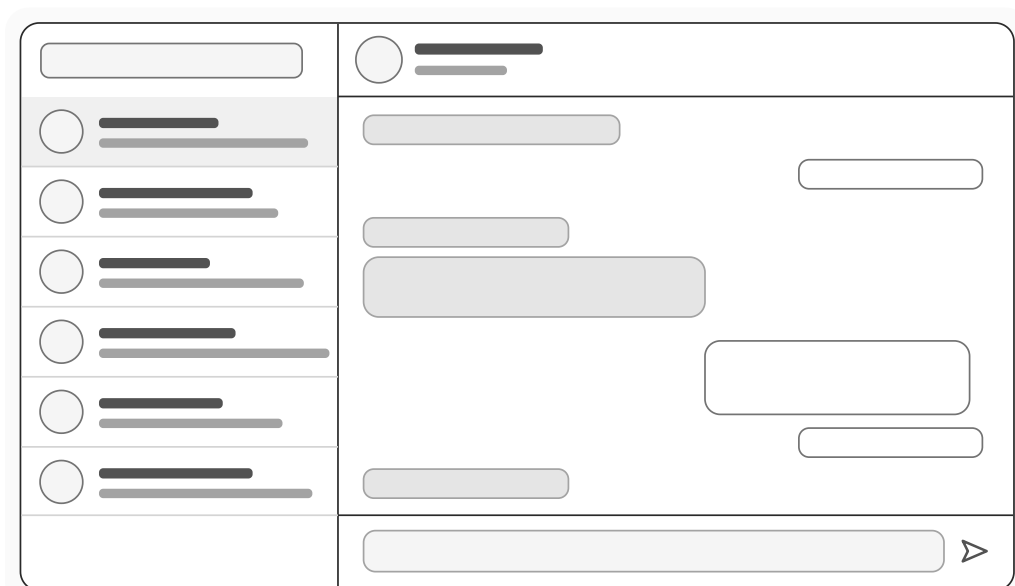


Canva

Question

Design a browser-based 1:1 chat application that supports text messages, real-time delivery, optimistic sending, offline queuing, delivery states, and reconnect catch-up. Assume one active tab per account for the core design.

Group conversations, rich media, calls, search, push notifications, end-to-end encryption, and coordinated multi-tab ownership are potential follow-ups rather than core requirements.



Real-life examples

These products demonstrate the messaging, delivery-state, and conversation-navigation patterns in scope.

- Messenger
 - WhatsApp Web
 - Slack
 - Discord
 - Telegram
-

Requirements exploration

Start by fixing the reliability contract. This design covers a browser-based 1:1 text chat with real-time delivery, optimistic sends, offline queuing, delivery states, and reconnect catch-up.

The core journey is deliberately small:

- Open a conversation and read cached history.
- Send a text message and see it immediately.
- Receive messages without refreshing.
- Distinguish sending, sent, delivered, read, and failed states.
- Queue sends while offline and reconcile them after reconnecting.

Assume one active tab per account. Group conversations, rich media, calls, search, push notifications, end-to-end encryption, and coordinated multi-tab ownership are potential follow-ups.

The most important non-functional requirements are low perceived latency, no lost user intent, deterministic message ordering, and recovery from duplicated or replayed network events.

Exercise Beta

Before evaluating WebSocket, SSE, and local-storage technologies for a browser chat, which missing product guarantees must be explicit? Select all that apply.

- ☐ 1 Which conversation types and message formats are core, and must users browse or send while offline?
- ☐ 2 Must the baseline use a managed WebSocket service, or can it combine SSE with HTTP sends?
- ☐ 3 What ordering and delivery guarantees are required, which retries must be idempotent, and which failures need visible recovery controls?
- ☐ 4 How should multiple devices behave, and is coordinated ownership across browser tabs part of the baseline or a follow-up?



Reveal answer

Check answer ↩

Architecture / high-level design

The chat client is local-first: the UI reads from a persistent client database, while a Data Syncer reconciles that database with the server. The real-time connection is a transport, not the UI's source of truth.

★ Write locally before using the network

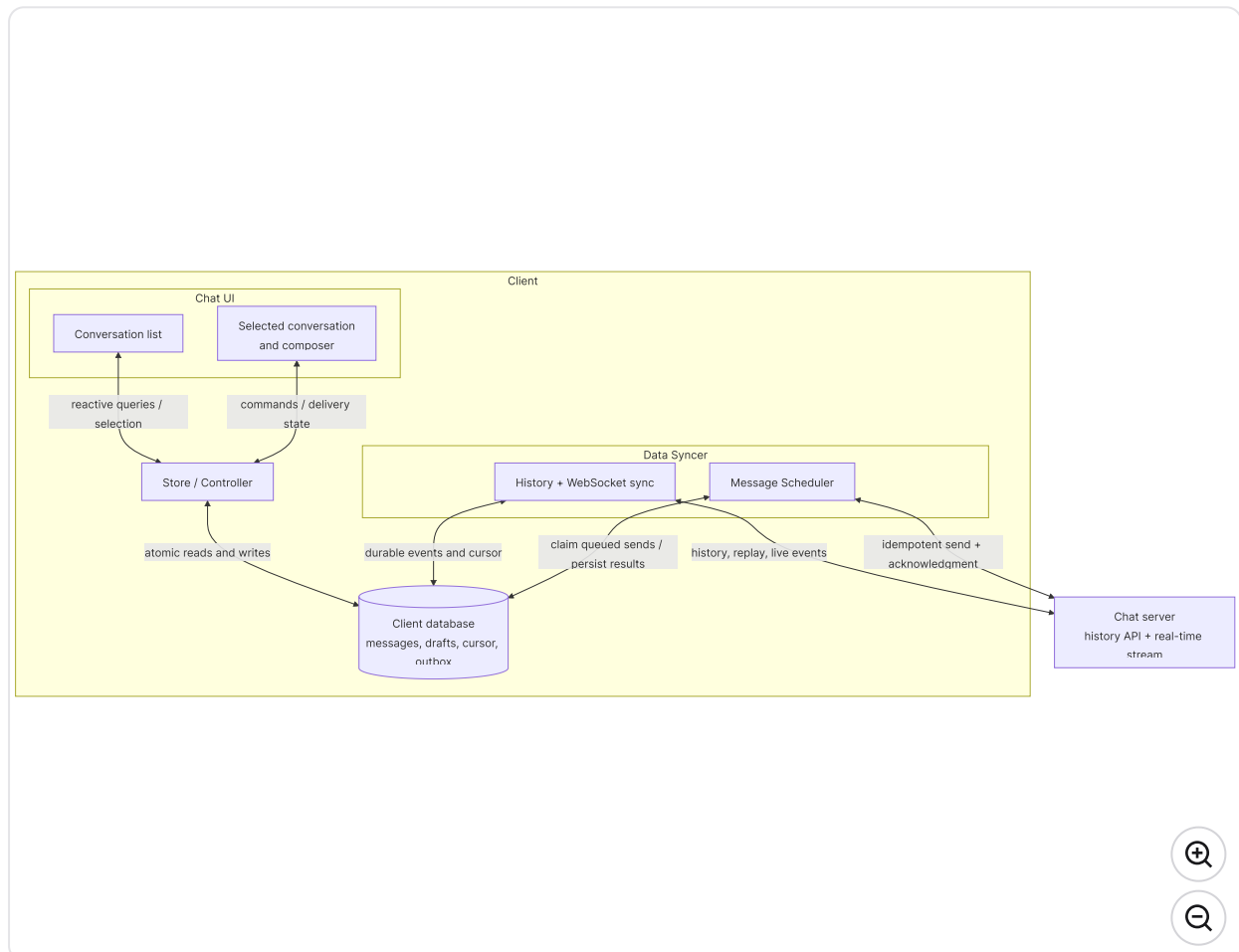
A message should appear immediately and survive a reload even when the network is unavailable. Persist the optimistic message and its outbox work first, then let the Data Syncer send and reconcile it asynchronously.

The main components are:

- **Chat UI:** Renders the Conversations List, selected conversation, composer, delivery states, and recovery controls.
- **Store/Controller:** Exposes reactive queries and short database transactions to the UI.

- **Client database:** Stores the account's cached conversations, messages, drafts, sync position, and outgoing requests.
- **Data Syncer:** Owns the WebSocket, reconnect loop, durable-event application, and Message Scheduler.
- **Message Scheduler:** Claims queued sends, preserves FIFO order within each conversation, retries transient failures, and exposes terminal failures.
- **Server:** Provides history APIs, an authenticated real-time event stream, idempotent message creation, and authoritative ordering.

The architecture diagram shows these boundaries and the data flow among the UI, local storage, synchronization components, and server.



Local-first chat client architecture

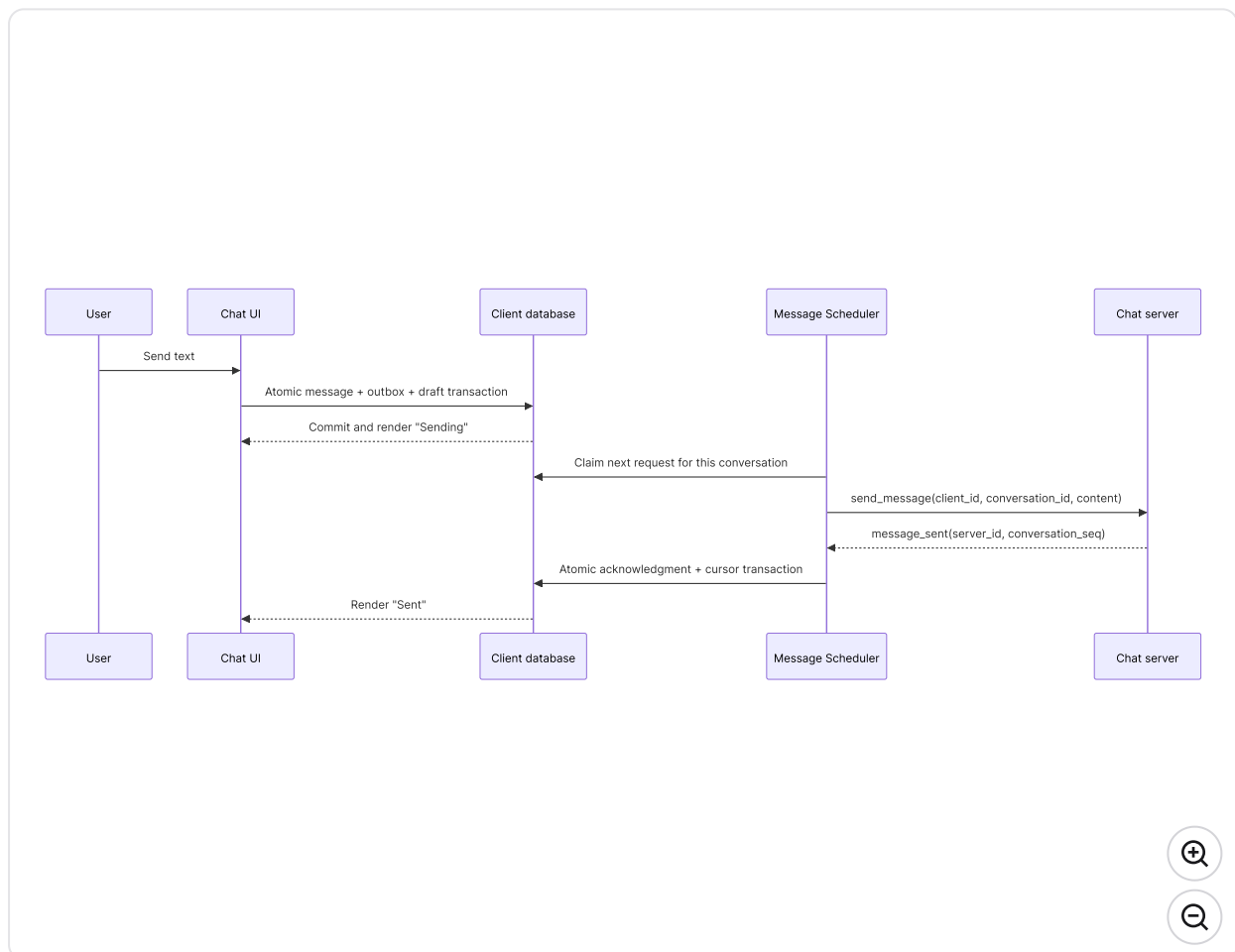
Sending a message

The synchronous user action ends at the client database. One IndexedDB **readwrite** transaction spans the **Message**, **SendMessageRequest**, and **DraftMessage** stores:

1. Generate a stable `local_id`, reuse it as the outgoing `client_id`, and insert a `Message` with `status: "sending"`.
2. Insert a `SendMessageRequest` that references the same `local_id`.
3. Delete the matching saved draft.
4. Publish the database-change notification and render the message only after the transaction commits.

If the transaction aborts, none of the three mutations take effect. The composer keeps its text and shows a recoverable local-storage error.

The network portion happens later and can be retried without changing the optimistic row's identity.



Optimistic send and acknowledgment

Receiving and reconnecting

The Data Syncer opens an authenticated WebSocket and subscribes with the account's last committed event sequence. The server first replays missed durable

events, then streams live events over the same connection.

Each durable event is applied in one database transaction together with every affected row, derived conversation metadata, and the new account cursor. A crash therefore cannot commit an event without its cursor or advance the cursor without its data.

The client reconnects with capped exponential backoff after a dropped socket. Replayed events are expected: upsert by durable identity, ignore already-committed event sequences, and request catch-up rather than guessing if a gap appears.

Rendering strategy

Use client-side rendering for the authenticated chat application. Messages are private, highly interactive, and not search-engine content, so server rendering adds limited value compared with a small application shell that hydrates from IndexedDB and then reconnects.

Exercise 1 of 3 Beta

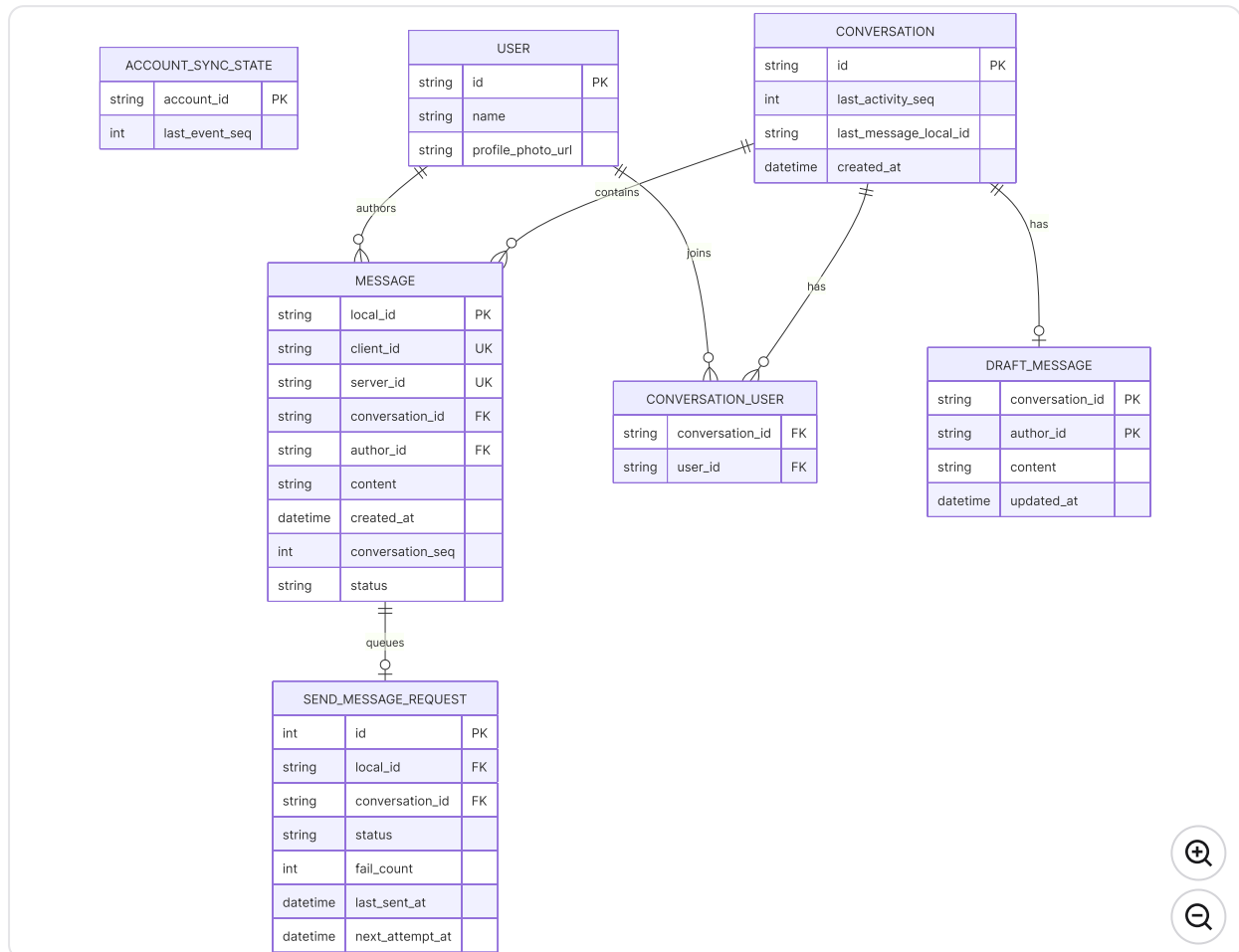
A signed-in chat client must show cached conversations while offline, render a sent message immediately, and reconcile it after reconnecting. Which architecture best supports those requirements?

- 1 Use component memory as the UI source, persist only server-confirmed messages, and let each conversation view call the transport directly.
- 2 Use server-rendered conversation snapshots as the UI source and refetch after each local send or streamed event.
- 3 Use the WebSocket event buffer as the UI source, adding IndexedDB only as a periodically refreshed read cache.
- 4 Use a persistent local database as the reactive UI source, with store transactions and a separate syncer owning transport, retries, and reconciliation.

[Reveal answer](#)[Check answer ←](#)

Data model

The data model separates stable local identity, server identity, per-conversation message order, and the account-level event cursor. Those values solve different problems and should not be overloaded.



Message identity and ordering

Every message has an immutable `local_id`, which is the IndexedDB primary key and React key for the row's lifetime. For an outgoing message, the client generates this value before the transaction and also sends it as `client_id`. For a message first learned from history or an incoming event, the client can derive a stable local value from `server_id`.

The server later supplies a nullable, unique `server_id`; acknowledging an optimistic message updates the existing row rather than replacing it. Keeping `local_id`, `client_id`, and `server_id` separate avoids requiring incoming messages to have a sender-scoped idempotency key.

The server also assigns `conversation_seq` transactionally when it accepts a message. Confirmed messages are ordered by `(conversation_id, conversation_seq)`, never by sender time or arrival order. Pending messages have no `conversation_seq`; render them after confirmed messages in their local enqueue order until acknowledgment places them canonically.

`created_at` remains useful for display, but client clocks cannot establish identity or ordering.

Conversation ordering

Each durable account event has an `event_seq`. An accepted or incoming message updates `Conversation.last_activity_seq` from that authoritative account sequence, so devices converge on the same conversation-list order for that account. Delivery and read receipts still advance the account cursor but do not reorder the Conversations List.

A local pending send may temporarily surface its conversation using a separate in-memory or client-only `pending_activity_at` value. It must not overwrite the authoritative `last_activity_seq`; acknowledgment replaces the provisional placement.

Drafts, outgoing requests, and failure states

`DraftMessage` is unique by `(conversation_id, author_id)` and persists text across reloads. Writes can be debounced or flushed on blur rather than performed on every keystroke.

`SendMessageRequest` references the stable `local_id` and stores the conversation, retry count, last attempt, next attempt, and one of these states:

- `pending` : Ready to send when the connection is healthy.
- `in_flight` : Sent and waiting for an acknowledgment.
- `retry_wait` : A transient failure is waiting for backoff.
- `terminal` : Automatic retries ended; the user must retry, copy, or discard the message.

The request and original message content remain available in the terminal state. Never silently delete old or exhausted sends. An explicit retry reuses the same `client_id`; discard is the only action that removes the user's intent.

Account boundary

Use a separate IndexedDB namespace for each authenticated account, such as `chat:<account_id>`. The sync position, cached entities, drafts, and outbox all live inside that namespace.

On account change, stop the old Data Syncer before opening the new namespace. On logout, clear that account's local database by default; any retained encrypted offline cache requires an explicit product and reauthentication policy.

Exercise 1 of 3 Beta

A message may be rendered optimistically, survive a reload, receive a server ID, and later be replayed during reconnect. Which data-model decisions preserve its identity and ordering through that lifecycle? Select all that apply.

- ☐ 1 Replace the optimistic row's local primary key with the server message ID when acknowledgement arrives, then order it by the sender's timestamp.
- ☐ 2 Persist outgoing request status, retry count, and last-attempt time in a client-side outbox.
- ☐ 3 Persist outgoing messages only after the first network attempt; keep queued offline messages and drafts in component-local state until then.
- ☐ 4 Give every message row an immutable local ID. For outgoing messages, also retain the client-generated idempotency ID after the server assigns its durable ID.
- ☐ 5 Store a server-assigned per-conversation sequence for canonical message ordering and a separate account event cursor for reconnect replay.

[Reveal answer](#)[Check answer ↩](#)

Interface definition (API)

The client uses HTTP for paginated reads and one WebSocket for live commands and account events. Both surfaces use the same authenticated account and conversation-authorization rules.

Send command and server idempotency

Every send attempt carries the same immutable request identity and payload.

```
1  {
2    "type": "send_message",
3    "client_id": "tmp_01HW...",
4    "conversation_id": "c_01HW...",
5    "content": "See you at 7!"
6  }
```

The server enforces uniqueness on `(sender_account_id, client_id)`. It atomically assigns the conversation sequence and stores the message. If an acknowledgment is lost, an identical retry returns the original `server_id` and `conversation_seq`; reusing the same `client_id` with different content fails validation.

To preserve a sender's intended order, the scheduler permits only one in-flight send per conversation while allowing sends to different conversations in parallel. It does not start the next send for a conversation until the prior request is acknowledged or the user explicitly discards it. A terminal failure remains visible at the head of that conversation's queue, where the user can retry or discard it without a later message overtaking it.

The acknowledgment is a normal durable event.

```
1  {
2    "type": "message_sent",
3    "event_seq": 48213,
4    "payload": {
5      "client_id": "tmp_01HW...",
6      "server_id": "m_01HW...",
7      "conversation_id": "c_01HW...",
```

```
8      "conversation_seq": 817
9    }
10 }
```

Applying it updates the optimistic row, advances the conversation metadata and account cursor, and removes the successful outbox request in one IndexedDB transaction.

Permanent validation or authorization failures move the request and message to a terminal failed state immediately. Timeouts and rate limits use capped exponential backoff; exhausting the cap produces the same visible state with Retry, Copy, and Discard actions.

Exercise 1 of 2 Beta

An optimistic message and its outbox request are already committed locally. The first send times out after the server may have accepted it. Which send and acknowledgment contract prevents a duplicate and reconciles the existing row?

- 1 Reuse a client ID only within one socket connection, then generate a replacement ID whenever reconnect creates a new session.
- 2 Reuse one client ID and identical payload on every attempt; enforce sender-scoped uniqueness and acknowledge the client ID, server ID, and conversation sequence.
- 3 Hash the message text and device timestamp on each attempt; deduplicate matching hashes and acknowledge only the server ID.
- 4 Use the client ID as the durable server ID and use device creation time as the message's final conversation position.

[Reveal answer](#)[Check answer ↩](#)

Local indexes

The hot client queries require a small set of deliberate indexes:

- `Message(conversation_id, conversation_seq)` for ordered history reads.

- Unique `Message(client_id)` for reconciling acknowledgments with this account's optimistic sends.
- Unique `Message(server_id)` for replay deduplication after acknowledgment.
- `SendMessageRequest(status, next_attempt_at, id)` for scheduler polling.
- `Conversation(last_activity_seq)` for the server-authoritative conversation list.
- Unique `DraftMessage(conversation_id, author_id)` for account-isolated drafts.

Native IndexedDB indexes support key lookups and ordered ranges, not full-text search. Text search needs a separate token index or a deliberately bounded scan and belongs in the follow-up discussion.

Exercise Beta

A local chat database contains millions of messages across conversations. Which indexes match the hot client queries? Select all that apply.

- 1

Index messages by client-generated ID.

☐
- 2

Index messages by conversation ID and server-assigned conversation sequence.

☐
- 3

Index messages by conversation ID and client-created timestamp, using that timestamp for both optimistic and confirmed ordering.

☐
- 4

Index outgoing requests by status, next-attempt time, and stable queue order.

☐

[Reveal answer](#)[Check answer ↩](#)

History APIs

History reads use opaque pagination cursors that are separate from the account event sequence.

- `GET /conversations?limit=N&cursor=<opaque>` returns conversations in authoritative activity order plus `next_cursor`.

- `GET /conversations/:id/messages?limit=N&before=<opaque>` returns older messages plus `next_cursor`.

The first endpoint hydrates the Conversations List. The second only loads older history; reconnect catch-up comes from the durable account event stream.

Durable event stream

Every server-originated durable mutation uses one per-account sequence namespace.

```
1  {
2    "type": "incoming_message",
3    "event_seq": 48214,
4    "payload": {
5      "server_id": "m_01HX...",
6      "conversation_id": "c_01HW...",
7      "conversation_seq": 818,
8      "sender_id": "u_01HW...",
9      "content": "See you then",
10     "created_at": "2026-04-22T19:11:02.410Z"
11   }
12 }
```

The account stream includes new messages, send acknowledgments, delivery and read receipts, edits, and tombstones. Every event advances `event_seq`, including events that do not create a message.

The client applies events with these rules:

1. Ignore an event at or below the last committed `event_seq`.
2. If the next sequence is missing, pause live application and request replay from the last committed sequence.
3. Upsert entities by durable identity and prevent delivery states from regressing.
4. Commit changed rows, derived conversation order, and `ACCOUNT_SYNC_STATE.last_event_seq` atomically.

Message history cursors never replace the account event cursor. This separation lets pagination move backward through one conversation while synchronization moves forward across every durable account mutation.

Use `wss:`, authenticate with a short-lived credential, and authorize every conversation-scoped command and history request. Render text messages as text rather than injecting HTML. More detailed socket hardening is a potential follow-up.

Exercise 1 of 4 Beta

A chat client is loading older messages in one conversation while durable message and receipt events continue arriving for the account. How should it track progress for those two flows?

- 1 Use one numeric offset that advances for live account events and moves backward whenever an older history page loads.
- 2 Keep separate cursor values, but advance the account-event cursor to the newest message returned by every successful history page.
- 3 Derive reconnect progress from the oldest and newest message sequences currently mounted in the selected conversation.
- 4 Use an opaque conversation-history cursor for pagination and a separate committed account-event cursor for reconnect.

[Reveal answer](#)[Check answer ↩](#)

Optimizations and deep dive

The core design is complete once optimistic writes, idempotent sends, deterministic ordering, and replay are coherent. The remaining sections focus on front-end behavior that commonly matters in an interview.

Typing indicators

Typing indicators are ephemeral rather than durable. Keep them in memory, omit them from the account event cursor, and do not retry or replay them after reconnecting.

Throttle `typing_start` on the sender and emit `typing_stop` on idle, blur, or send. On the receiver, keep a short expiry timer so a dropped stop event cannot leave the indicator visible indefinitely. Dropped, duplicated, or reordered typing events are acceptable because the state is only a temporary hint.

Exercise Beta

A typing-stop event is lost when a recipient briefly disconnects. Which policies let the indicator recover without giving it durable-message semantics? Select all that apply.

- ☐ 1 Keep the latest typing state in a short-lived server cache and replay it when a recipient reconnects within the expiry window.
- ☐ 2 Keep current typing state in memory and expire it on a short receiver-side timer.
- ☐ 3 Allow dropped, duplicated, or out-of-order typing events without replaying them after reconnect.
- ☐ 4 Persist typing-stop events until every recipient device acknowledges them or its session expires.

[Reveal answer](#)[Check answer ↩](#)

Network recovery

Connection state should be explicit: `connecting`, `connected`, `reconnecting`, and `offline`. A healthy WebSocket heartbeat is the authoritative signal; `navigator.onLine` is only a hint because it cannot prove the chat server is reachable.

The recovery policy is:

- Pause outbox claims while the socket is unhealthy.
- Reconnect with capped exponential backoff and jitter.
- Resume from the last committed account event sequence.

- Retry transient sends with the same `client_id`.
- Keep one in-flight send per conversation and bounded parallelism across conversations.
- Stop automatic retries at a visible terminal state rather than retrying forever or deleting the request.

Exercise Beta

A mobile chat session repeatedly loses connectivity. Which behaviors create a resilient send and reconnect path? Select all that apply.

- 1 Flush every queued send as soon as a replacement socket opens, then request missed account events in the background. ☐
- 2 Pause outbox claims until a heartbeat confirms the chat connection is healthy. ☐
- 3 Treat `navigator.onLine === true` as proof that the chat service is reachable and immediately mark the connection healthy. ☐
- 4 Resume from the last committed cursor, apply the missing replay, and only then resume queued sends. ☐
- 5 Reconnect and retry transient failures with capped exponential backoff and jitter. ☐

[Reveal answer](#)[Check answer ↩](#)

Performance

Chat performance depends on bounding work at the UI, storage, and network boundaries.

- Virtualize long message histories and preserve a small overscan window.
- Key every bubble by immutable `local_id`. Status, content, and reactions are props, not parts of the key.
- Update only the appended or changed bubble and the affected conversation row.

- Batch IndexedDB reads through the `(conversation_id, conversation_seq)` index instead of issuing one read per message.
- Load older history in pages and lazy-load heavy composer tools.
- Reserve fixed media dimensions so resolved assets do not shift the scroll position.

Exercise Beta

A conversation with years of history becomes slow whenever one new message arrives. Which change most directly fixes the rendering hot path?

- 1 Mount a bounded message window keyed by stable local IDs, and patch only the changed bubble and affected conversation row.
- 2 Virtualize the visible window but key bubbles by their current array index and rebuild the window whenever older history is prepended.
- 3 Reload and replace the selected conversation snapshot from IndexedDB after every incoming event or delivery-state change.
- 4 Keep the complete history mounted and memoize each bubble using a serialized copy of its full message record.

[Reveal answer](#)[Check answer ↩](#)

Accessibility

Incoming content must remain discoverable without interrupting composition or moving focus.

- Use a labeled `role="log"` for live message appends. Suppress announcements while hydrating cached content or prepending older history so a screen reader does not narrate the backlog.
- Announce routine new messages politely. Reserve assertive announcements for failures such as a send error or lost connection.
- Keep focus in the composer while messages arrive. Modal tools trap focus and restore it to their trigger when closed.

- On desktop, Enter may send and Shift+Enter may insert a newline, but never send while `KeyboardEvent.isComposing` is true. Provide a labeled Send button and define an appropriate mobile `enterkeyhint` behavior.
- Do not encode delivery states by color alone, and honor `prefers-reduced-motion`.
- Treat 44 by 44 CSS pixels as a generous product target; WCAG 2.2 AA permits a 24 by 24 CSS-pixel minimum with exceptions.
- Use logical CSS properties for RTL layout and determine text direction per message. Whether sender ownership sides mirror is a product convention, not an automatic consequence of RTL text.

Exercise 1 of 2 Beta

On reconnect, a chat prepends cached history, receives new messages, and shows a connection error while the user is typing. Which behaviors communicate the changes without replaying history or disrupting input? Select all that apply.

- 1

Announce the connection failure through a programmatic status and expose a keyboard-reachable recovery control without moving composer focus.

☐
- 2

Move focus to the reconnect banner when it appears, then restore focus to the composer after connectivity returns.

☐
- 3

Show a red failed icon beside the message and put the detailed error in a tooltip reachable from that icon.

☐
- 4

Append incoming messages to a labeled chat log with polite announcements, while suppressing announcements during cache hydration and history prepends.

☐

[Reveal answer](#)[Check answer ↩](#)

Offline app behavior

IndexedDB makes cached conversations and pending sends available without a network connection. A service worker may cache the versioned application shell and static assets, but API responses and the WebSocket remain network-only.

The correctness baseline does not depend on background execution: the outbox survives closing the tab and the normal Message Scheduler resumes it on the next launch. A closed-tab Background Sync transport is optional and requires a separate idempotent HTTP mutation path, described under potential follow-ups.

Exercise Beta

A send request with a stable client ID is already persisted when the user closes the offline app. On the next launch, which baseline resumes it without requiring closed-tab background execution?

- 1 Mark every previously in-flight request as sent because the server might have accepted it before the app closed.
- 2 Copy each interrupted request into a new outbox entry with a new client ID before resuming the scheduler.
- 3 Treat a service worker that delivers every request while the app is closed as a prerequisite for preserving queued messages.
- 4 Rehydrate pending and interrupted requests, reconnect and catch up, then let the normal scheduler retry the conversation head with its original client ID.

[Reveal answer](#)[Check answer ↩](#)

Navigation and scroll position

Appending a live message and prepending older history require different scroll behavior.

If the reader is already pinned to the bottom, keep the view pinned as messages arrive. Otherwise preserve the visible content and show a new-message or Scroll to Bottom affordance. When older history is prepended, compensate for the inserted height or anchor to the first previously visible message.

Use fixed-size media placeholders, because late image sizing can otherwise move the viewport. Preserve a per-conversation scroll anchor in session state, and place an unread divider at the first unread message when opening a conversation.

Exercise Beta

A user has scrolled up to read older messages when new messages arrive below and older history is prepended above. What should the interface do?

- 1 Anchor the visible content through prepends and show a new-message affordance instead of moving the reader to the bottom.
- 2 Commit incoming messages to storage but leave the rendered list unchanged until the reader returns to the bottom.
- 3 Restore the same visible array index after a prepend and allow late-loading media above it to use intrinsic dimensions.
- 4 Keep the same numeric `scrollTop` after prepending older rows, and scroll to the bottom for each live message.



Reveal answer

Check answer ↩

Potential follow-ups

These production concerns are useful interviewer follow-ups, but they should not displace the core send and sync design in a 30-minute answer.

- **Coordinated multi-tab ownership:** Let every tab write short IndexedDB transactions, but elect one network owner with Web Locks and notify peers with `BroadcastChannel`. A production design also releases resources on freeze or `pagehide` and uses a generation token so an old leader cannot resume beside a new one.
- **Very stale clients:** When the server can no longer replay from an old cursor, fetch a bounded recent snapshot. Preserve the complete unsent mutation closure—request, optimistic message or self-contained payload, conversation/account context, and attachment references—then revalidate the account and conversation before replay.
- **Closed-tab sending:** Background Sync cannot reuse a page-owned WebSocket. It needs an authenticated, idempotent HTTP send endpoint with the same `client_id`, ordering, result, and outbox-claim rules as the socket path, plus coordination when a page reopens.

- **Search:** IndexedDB has no native full-text index. Maintain a token or inverted index, usually in a worker, or scan only a bounded local window and use server search for nonresident history.
- **Push notifications:** Ask for notification permission from a visible user action, then create the Push subscription. Handle denial and revocation without weakening in-app delivery.
- **Attachments and binary data:** Upload bytes through resumable HTTP or object storage and send only the resulting reference through the chat protocol. Do not block the shared real-time connection with large attachment frames.
- **WebSocket hardening:** Validate the handshake origin, close sockets on logout or token expiry, authorize every action, validate event schemas and payload sizes, and rate-limit by account and connection.
- **Protocol evolution and product extensions:** Use explicit event types and tolerate unknown optional fields. Reactions are durable idempotent events; disappearing-message expiry is server-enforced and reconciled with tombstones rather than independent client timers.
- **Incoming flow control:** Cap frame and sync-page sizes, decode and apply through a bounded queue, and reconnect from the last committed cursor if the client cannot keep up.

Summary

The core design is a local-first chat client with one persistent database and one Data Syncer. The UI commits an optimistic `Message`, its `SendMessageRequest`, and draft deletion atomically, then renders from IndexedDB. The Message Scheduler preserves FIFO order within each conversation and retries with the same `client_id`; the server enforces idempotency and assigns the durable `server_id` and `conversation_seq`.

Synchronization uses a separate per-account `event_seq`. Every durable event, affected entity, derived conversation order, and cursor advancement commits in one transaction. History pagination has its own opaque cursor, so loading older messages cannot corrupt reconnect state. The immutable `local_id` also gives React a key that survives acknowledgment and delivery-state changes.

For a concise interview answer, start with the local-first boundary, atomic optimistic send, idempotent server contract, per-conversation ordering, and account-level replay.

Then cover virtualized rendering, accessible live updates, visible terminal failures, and scroll preservation. Multi-tab leadership, stale snapshots, closed-tab delivery, search, push, attachments, protocol evolution, and backpressure are follow-ups when the interviewer asks for production depth.

References

The following references provide deeper background for the transport, storage, synchronization, and accessibility decisions in this design.

Messaging architecture and synchronization

These case studies describe ordered update queues, client persistence, and reconnect behavior in large messaging systems.

- [Building mobile-first infrastructure for Messenger](#)
- [How Discord stores trillions of messages](#)
- [Airbnb's approach to client-side messaging sync](#)

Browser platform primitives

These platform documents define the client storage, real-time transport, and accessibility semantics used above.

- [Indexed Database API 3.0](#)
- [The WebSocket Protocol](#)
- [Web Locks API](#)
- [Broadcast Channel API](#)
- [Using `role="log"` for sequential updates](#)
- [UI Events composition behavior](#)

Security and optional background capabilities

These references cover the safeguards and browser APIs intentionally left as follow-up depth.

- [OWASP WebSocket Security Cheat Sheet](#)
- [Background Synchronization API](#)
- [Push API](#)

Exercises

Exercise 1 of 21 Beta

Before evaluating WebSocket, SSE, and local-storage technologies for a browser chat, which missing product guarantees must be explicit? Select all that apply.

- 1

Which conversation types and message formats are core, and must users browse or send while offline?

☐
- 2

Must the baseline use a managed WebSocket service, or can it combine SSE with HTTP sends?

☐
- 3

What ordering and delivery guarantees are required, which retries must be idempotent, and which failures need visible recovery controls?

☐
- 4

How should multiple devices behave, and is coordinated ownership across browser tabs part of the baseline or a follow-up?

☐



Reveal answer

Check answer ↩

Discussions

↶ ↷ | Normal T_T | B I U ↻ | ✎ <> Aa ☰ ☷ </> ☰

Type comment here...

0/1000

Post

[🔔 Report an issue](#)[✓ Mark complete](#)

Subscribe to our newsletter

[Notify me](#)

Practice

[Get started](#)[JavaScript functions](#)[User interface coding](#)[System design](#)[Quiz](#)

Guides

[Front End Interview Playbook](#)[Front End System Design Playbook](#)[React Interview Playbook](#)[Behavioral Interview Playbook](#)

Study Plans

[1 Week Plan](#)[1 Month Plan](#)[3 Months Plan](#)

Company

[Pricing](#)[Promotions](#)[Roadmap](#)[About](#)[Team](#)[Contact us](#)[Become an affiliate](#)[Careers](#)[Blog](#)[Medium](#)[DEV Community](#)[English \(US\) ▾](#)

